

Documentación de Proyecto - Cafetería Plus

Programación de Servicios y Procesos

Cristian Fernández

3 de diciembre de 2025

Índice

1. Descripción del problema	2
2. Patrón de diseño Productor/Consumidor	3
3. Nuevas clases	4
4. Requisitos funcionales	6
5. Requisitos no funcionales	7
6. Casos de uso	8
7. Historias de usuario	9
8. Herramientas de desarrollo	10
9. Lenguajes de programación	11
10. Interfaz de la aplicación	12

1. Descripción del problema

En esta ocasión recuperamos el proyecto anterior en el que simulábamos el funcionamiento de una cafetería mediante distintos hilos de ejecución con objeto de expandir su funcionalidad mediante la implementación de una capa extra de complejidad, gracias a la cual seguiremos profundizando en el estudio de la sincronía de procesos.

Los puntos clave que nos permitirán dicha tarea serán la incorporación de los **métodos *synchronized*** junto con el uso del **patrón de diseño Productor/Consumidor**.

El programa debe seguir estas pautas:

- Crear una clase Barista (Productor).
- Crear dos clases Camarero (Consumidor).
- Crear una clase Cafetera (Búfer Compartido).
- Crear clases Cliente mediante la pulsación del botón '*Dar Paso*'.
- Cada cliente debe intentar pedir un café y esperar el tiempo especificado. Los camareros deben esperar a que haya cafés disponibles y si es así servirlos en orden de llegada de la clientela. El barista debe hacer cafés incansablemente hasta que la cafetera no admita más (búfer lleno). Si un cliente se va antes de que su café esté listo, el camarero debe continuar con el siguiente cliente.
- El programa debe continuar ejecutándose hasta que se pulse el botón '*Finalizar*'.

2. Patrón de diseño Productor/Consumidor

El patrón de diseño Productor/Consumidor desacopla los procesos que producen datos de aquellos que los consumen, utilizando un búfer compartido para almacenar datos temporalmente. Este patrón es ideal para aplicaciones con tareas asincrónicas e independientes que se ejecutan a diferentes velocidades, mejorando la escalabilidad y el rendimiento al permitir que el productor y el consumidor trabajen en paralelo.

Sus componentes clave son:

- **Productor:** Es un hilo o proceso que genera datos y los añade a una estructura de datos compartida, como una cola o búfer.
- **Consumidor:** Es un hilo o proceso que retira los datos del búfer y los procesa.
- **Búfer Compartido:** Es una estructura de datos (ej. una cola) que actúa como intermediario, almacenando los datos hasta que el consumidor esté listo para procesarlos.
- **Sincronización:** Mecanismos para controlar el acceso al búfer y asegurar que los productores no añadan datos a un búfer lleno y los consumidores no intenten retirar datos de un búfer vacío.

3. Nuevas clases

A continuación unas capturas de las nuevas clases incorporadas al proyecto.

```
class Barista extends Thread { 4 usages  cristianCmon
    private Cafetera cafetera; 2 usages

    public Barista() {}; 2 usages  cristianCmon

    public Barista(Cafetera c) { this.cafetera = c; }

    private int getPosicionCafe() { 1 usage  cristianCmon
        int posicion = -1;

        for (int i = 0; i < imagenes.size(); i++) {
            if (!imagenes.get(i).isVisible()) {
                posicion = i;
                break;
            }
        }

        return posicion;
    }

    @Override  cristianCmon
    public void run() {
        long tiempoPreparacionCafe = (long)(Math.random() * 3000) + 2000; // de 3 a 5 segundos
        int posicionCafe;

        do {
            cambiarColor( barista: this, color: "naranjaPuntos");

            try {
                System.out.println("Barista interactúa con cafetera...");
                join(tiempoPreparacionCafe);
                posicionCafe = getPosicionCafe();
                cafetera.put(posicionCafe);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            System.out.println("Café LISTO...");
        } while (true);
    }
}
```

Figura 1: Clase Barista

```

class Cafetera { 6 usages  🧑 cristianCmon
    int cafe; 6 usages
    int cantidad = 0; 4 usages
    private boolean disponible = false; 4 usages

    public synchronized int get() { 1 usage  🧑 cristianCmon
        while (!disponible) {
            try {
                System.out.println("Camarero esperando cafetera...");
                wait();

            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        if (cantidad > 0) {
            cafe = cogerPrimerCafeDisponible();
        }

        if (cafe != -1) {
            System.out.println("Sirviendo café...");
            disponible = estaCafeteraLlena();

            cantidad--;
            notifyAll();
        }

        return cafe;
    }

    public synchronized void put(int valor) { 1 usage  🧑 cristianCmon
        while (disponible) {
            try {
                System.out.println("cafetera llena, barista en espera...");
                cambiarColor(new Barista(), color: "naranja");
                wait();

            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        cafe = valor;

        if (cafe != -1) {...}
    }

    private boolean estaCafeteraLlena() { return cantidad > 3; // posiciones cafetera 0-3 }

    private int cogerPrimerCafeDisponible() {...}
}

```

Figura 2: Clase Cafetera

4. Requisitos funcionales

A continuación se listan los requisitos funcionales del sistema:

RF-1: Ver una leyenda de colores que informen del estado de los Hilos.

RF-2: Pulsar un botón que inicie la ejecución del programa.

RF-3: Mostrar dinámicamente todo el proceso de ejecución.

RF-4: Pulsar un botón que finalice la ejecución del programa.

5. Requisitos no funcionales

A continuación se listan los requisitos NO funcionales del sistema:

RNF-1: El tiempo de respuesta al pulsar el botón Iniciar no debe superar los 2 segundos.

RNF-2: La interfaz gráfica de usuario ha de ser fácilmente entendible.

RNF-3: Cada acción realizada por los hilos debe ser lo suficientemente descriptiva para el usuario.

6. Casos de uso

A continuación se muestran los casos de uso mediante la siguiente tabla:

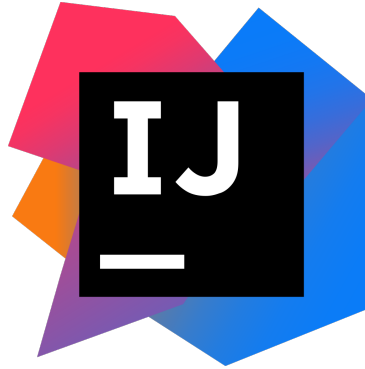
Caso de uso	Descripción
Consultar leyenda colores	El usuario consulta el funcionamiento de los hilos de la aplicación.
Iniciar aplicación	El usuario puede iniciar la aplicación pulsando un botón.
Terminar aplicación	El usuario puede finalizar la ejecución pulsando un botón.

7. Historias de usuario

A continuación se muestran las historias de usuario mediante la siguiente tabla:

ID	Como...	Quiero...	Para...
HU-01	Usuario	Pulsar un botón	Iniciar la aplicación
HU-02	Usuario	Entender el comportamiento de los hilos	Deducir cómo funciona la concurrencia
HU-03	Usuario	Pulsar un botón	Finalizar la ejecución de la aplicación

8. Herramientas de desarrollo



IntelliJ Community Edition es un IDE gratuito y de código abierto, ideal para el desarrollo en lenguajes JVM y Android. Sus características principales incluyen un potente editor de código con autocompletado inteligente y refactorización, soporte para control de versiones, un depurador integrado, y herramientas de pruebas unitarias. Está disponible para todas las plataformas.

Características principales:

- **Editor de código potente:** Ofrece resaltado de sintaxis, análisis en tiempo real, sugerencias de código, e inspecciones y correcciones rápidas.
- **Soporte para múltiples lenguajes:** Es compatible con Java, Kotlin, Groovy, Scala, y otros lenguajes JVM.
- **Integración con control de versiones:** Permite la integración con sistemas como Git, SVN, y Mercurial.

9. Lenguajes de programación



JavaFX se caracteriza por su capacidad de crear aplicaciones enriquecidas visualmente, integrando multimedia (audio, video) y gráficos vectoriales. Permite desarrollar aplicaciones multiplataforma (escritorio, web, móvil, smart TV). Otras características clave son la integración fluida con el ecosistema Java y herramientas como FXML y Scene Builder para un desarrollo más eficiente y colaborativo.

Características principales:

- **Interfaces gráficas de usuario (GUI) enriquecidas:** Permite crear interfaces modernas y atractivas con animaciones, gráficos vectoriales, y elementos personalizables.
- **Integración multimedia:** Facilita la inclusión de contenido de audio, video y otros medios dentro de las aplicaciones.
- **Desarrollo declarativo (FXML):** Utiliza FXML, un lenguaje basado en XML, para describir la interfaz de usuario, lo que facilita el trabajo colaborativo entre diseñadores y desarrolladores.

10. Interfaz de la aplicación

A continuación unas capturas de las diferentes vistas de la interfaz.



Figura 3: Vista inicial, muestra comportamiento hilos por código de colores



Figura 4: Cafetera cargada y barista/camareros libres



Figura 5: Servicio normal con carga de trabajo rutinaria